

Unidade I

1 QUALIDADE E PRODUTIVIDADE DE SOFTWARE

A **qualidade em software** refere-se ao grau em que um produto atende às **necessidades e expectativas dos usuários**, estando relacionada diretamente aos requisitos não funcionais (RNF) do software. O software de qualidade apresenta confiabilidade, usabilidade, eficiência, manutenibilidade, portabilidade e segurança.

A **produtividade em software** está relacionada à **eficiência do processo de desenvolvimento**, isto é, à **quantidade de produto** (funcionalidades, linhas de código, entregas) gerada em relação aos recursos utilizados (tempo, esforço, equipe e custo). Quanto **maior o valor que a equipe entrega em menos tempo e com menos recursos, maior é a produtividade**.

O equilíbrio ideal é buscar alta produtividade com qualidade sustentável, adotando boas práticas de engenharia de software, automação de testes e melhoria contínua de processos.

1.1 Demanda, qualidade e produtividade

A **qualidade de software** refere-se à **avaliação de um atributo próprio de um determinado componente de software**, de modo que ele atenda aos requisitos, seja livre de defeitos e entregue valor ao usuário final.

Um **produto de alta qualidade normalmente tem um custo elevado**, o que compromete sua competitividade no mercado; por outro lado, a redução da qualidade aumenta o número de defeitos e, consequentemente, o aumento do custo na logística reversa para manutenção, o que leva a um aumento no custo do produto.

Para atender à demanda de um determinado produto, a produção terá que acompanhar e, por vezes, reduzir o custo da produção para que o produto continue competitivo no mercado. Para aumentar a produção, é necessário aumentar o volume produzido. Para isso, deve-se melhorar a produtividade, o que implica em, por exemplo, produzir, no mesmo período ou conforme a necessidade, uma quantidade superior à do período anterior.

A produtividade de software mede a eficiência com que a equipe de desenvolvimento transforma requisitos funcionais (RF) em funcionalidades entregues.

Empresas enfrentam este **desafio: aumentar a produção em menor tempo a custos baixos. Para isso, abrem mão de alguns critérios de qualidade**, tais como suporte, inspeção, manutenção, testes, diagnósticos e outros.

1.2 Importância da qualidade no desenvolvimento ágil

A adoção de metodologias ágeis (como Scrum, XP, FDD, Kanban e outras), também chamadas de metodologias adaptativas, mudou o panorama do desenvolvimento de software, priorizando entregas contínuas e adaptação a mudanças.

Essa agilidade não deve ocorrer em detrimento da excelência técnica. Pelo contrário, a qualidade assume uma posição imperativa e intrínseca no desenvolvimento ágil.

No método ágil, a qualidade não é obtida na etapa final (como no modelo cascata), mas se constrói de forma contínua e integrada. Sem um rigoroso foco na garantia da qualidade do software (GQS) – do inglês, software quality assurance (SQA) –, a velocidade inicial rapidamente se transforma em débito técnico, metáfora para o custo futuro de refatoração e correção que se acumula quando o código de baixa qualidade é priorizado em detrimento do design.

1.2.1 Controle da qualidade: garantia de qualidade de software (GQS) ou software quality assurance (SQA)

A garantia da qualidade de software (GQS) é um conjunto sistemático de atividades planejadas e implementadas para garantir que os processos, produtos e artefatos de software estejam em conformidade com os padrões de qualidade estabelecidos pela organização e pelas normas internacionais.

SQA tem como objetivo assegurar que o software atenda aos requisitos especificados, minimizando defeitos e aumentando a confiabilidade e a satisfação do cliente.

Conceito de qualidade de software envolve a adequação do produto ao uso pretendido, sua conformidade com os requisitos do software e a capacidade do processo de desenvolvimento de produzir resultados consistentes e previsíveis.

Controle da qualidade, dentro do contexto da SQA, atua como um sistema de gestão que acompanha todas as fases do ciclo de vida do software



A qualidade desejada se refere à definição dos RNF do software, determinados para a sua construção. Os RNF correspondem às propriedades de qualidade e às restrições técnicas que influenciam o comportamento global de um sistema de software.

Principais RNF ISO 25000:

- **Funcionalidade:** é caracterizada por um conjunto coeso de operações relacionadas, desenvolvidas de acordo com a mesma base tecnológica e coerentes com a arquitetura do sistema.
- **Confiabilidade:** é a capacidade do software de manter seu desempenho e disponibilidade dentro de limites aceitáveis, mesmo sob condições imprevistas.
- **Usabilidade:** se refere à facilidade de uso, compreensão, atratividade, interação e aprendizado do software. A usabilidade é o principal critério a ser avaliado na interface humano-computador (IHC) e envolve aspectos de experiência do usuário (user experience – UX), acessibilidade e atratividade visual.
- **Segurança:** conjunto de práticas, medidas, mecanismos e políticas destinados à proteção do sistema de software contra ameaças, ataques e vulnerabilidades, internos e externos, contempla a confidencialidade, integridade, disponibilidade e autenticidade das informações.
- **Eficiência:** se refere ao desempenho do software e de suas funcionalidades, com relação a tempo de resposta, uso de recursos computacionais e capacidade de processamento.
- **Manutenibilidade:** corresponde à facilidade de implementar mudanças para atualizações do ambiente operacional, melhoria ou adaptação de funcionalidades a novos contextos.
- **Portabilidade:** refere-se a quanto

um determinado produto de software é portátil para outros ambientes operacionais, ou seja, o quanto pode ser adaptado ou executado em diferentes plataformas, ambientes operacionais ou dispositivos, com o mínimo de esforço técnico.

As eficazes práticas de desenvolvimento de software a seguir melhoram a relação entre qualidade e produtividade:

- Automação de testes: reduz o tempo de testes manuais e aumenta a cobertura de testes, melhorando a qualidade sem sacrificar a produtividade.
- Desenvolvimento ágil: promove entregas frequentes e iterativas, permitindo ajustes rápidos e contínuos que mantêm a qualidade alta e a produtividade constante.
- Reuso de código: São produzidos componentes reutilizáveis que podem acelerar o desenvolvimento e reduzir a probabilidade de erros.

Métodos na SQA:

- **Verificação e validação (V&V):** é uma das fases críticas do ciclo de desenvolvimento, porque **oferece garantias para que o software não apenas satisfaça as necessidades do usuário (validação), mas também esteja em conformidade com seus requisitos, especificações e padrões técnicos (verificação).**

Os testes de software são compostos por uma série de técnicas aplicadas durante todo o ciclo de desenvolvimento, que incluem:

- Teste unitário: tem foco nos componentes de código isolados.
- Teste de integração: verifica e testa a interface entre os módulos e componentes do sistema de software.
- Teste de sistema: se refere à validação dos RF e RNF do sistema de software completo.
- Teste de aceitação do usuário (user acceptance testing – UAT): assegura que o sistema atenda às necessidades do negócio e seja utilizável pelo cliente.

- **Análise estática e revisão de pares (peer reviews):** são atividades de SQA preventiva, projetadas para identificar e corrigir defeitos já no início do ciclo de desenvolvimento, quando o custo da correção é significativamente baixo.

- A análise estática se refere à inspeção do código, da documentação e dos modelos do sistema, para verificar se estão de acordo com os requisitos pré-definidos do software

— As revisões por pares são um processo sugerido pela SQA e por diversas metodologias ágeis, em que o trabalho do desenvolvedor, como código-fonte, documentação de requisitos ou modelos de arquitetura, é examinado por um ou mais colegas de trabalho.

— As revisões de código são revisões formais ou informais, auditorias, walkthroughs e inspeções para identificar problemas precocemente, incluindo inspeções (método estruturado baseado em papéis para detectar defeitos em artefatos, como código e documentação) e o uso de análise estática automatizada como complemento às inspeções manuais.

- **Gerenciamento de configuração de software (software configuration management – SCM):** o SCM é o processo de identificação, organização e controle sistemático de modificações nos artefatos do sistema durante o ciclo de vida. O SCM se baseia no controle de versão com ferramentas.

- **Gestão de artefatos e documentação:** artefato de trabalho que garante a rastreabilidade e a manutenibilidade do software. Inclui uma documentação técnica detalhada sobre a arquitetura de software, modelo de dados, especificações de requisitos e a documentação da interface de programação de aplicações (application programming interface – API).

- **Gestão de risco e plano de contingência:** essa atividade gerencial identifica, monitora, avalia e mitiga proativamente os riscos que ameaçam a qualidade e a agenda do projeto. — A mitigação de riscos consiste em definir regras e procedimentos de qualidade (como gateways de qualidade obrigatórios ou limites de cobertura de teste) para evitar a introdução de defeitos críticos. — Já o plano de contingência se refere à definição de ações de fallback (alternativa de segurança) e recursos a serem mobilizados quando um risco se materializa.

1.2.2 Custo da qualidade em engenharia de software e metodologias ágeis

Na engenharia de software, o custo da qualidade (cost of quality – CoQ) representa o conjunto de investimentos, esforços e recursos aplicados para garantir que o produto atenda aos requisitos do software, esse custo abrange todas as atividades voltadas à prevenção, avaliação e correção de falhas durante o ciclo de vida do software.

Nas metodologias ágeis, o custo da qualidade é entendido como parte do valor entregue ao cliente, ou seja, o equilíbrio entre o esforço investido em qualidade e o retorno percebido em confiabilidade, desempenho e satisfação do usuário final.

Scrum ou DevOps, a qualidade é continuamente avaliada por meio de iterações curtas, feedbacks constantes, testes automatizados e monitoramento contínuo.

Custo da qualidade pode ser entendido como o preço justo que o cliente esteja disposto a pagar. Esses custos podem ser divididos em:

- Custos de conformidade: são os custos necessários para assegurar que o software atenda aos padrões e requisitos de qualidade estabelecidos pela equipe e partes interessadas. Incluem: — Custos de prevenção: planejamento, revisões técnicas contínuas, projeto, definição de padrões de código, ferramentas de diagnóstico e testes, além de treinamento da equipe de desenvolvimento e dos usuários. Exemplo ágil: refinamento de backlog, sessões de pair programming e retrospectivas de sprint contribuem diretamente para a prevenção de defeitos. — Custos de avaliação: são os esforços para inspeções intra e interprocessos, produção, modelos de testes e diagnósticos, precisão, manutenção dos equipamentos e plano de mudanças no software. Exemplo ágil: revisões de incremento e testes de regressão automatizados em cada ciclo reduzem o risco de falhas não detectadas.
- Custos de não conformidade: representam os custos associados à falta de qualidade, como falhas, retrabalho ou defeitos que são identificados tardiamente. — Custos de falha interna: revisão de código, engenharia reversa, análise do modo como a falha ocorreu (root cause analysis, ou análise da causa-raiz), refatoração de código, atrasos na entrega e reparação de defeitos. Exemplo ágil: falhas identificadas em testes automatizados durante a integração contínua (pipeline de continuous integration – CI) exigem correção imediata antes da próxima integração. — Custos de falha externa: solução das queixas, devolução e substituição do produto, incidentes de segurança, manutenção da linha de suporte e serviços da garantia de qualidade e índices de satisfação. Exemplo ágil: após o report de um bug por parte de usuários, em fase de implantação, houve custos adicionais para a correção devido a análises de reversão, afetando, assim, o fluxo de valor.

1.3 Fatores, atributos e métricas da qualidade do software

Para determinar a qualidade, três conceitos distintos são utilizados: fator de qualidade, atributo de qualidade e métrica de qualidade.

1.3.1 Fator de qualidade

O fator de qualidade é o conjunto de uma ou mais características que influenciam a qualidade do produto de software. Um determinado fator de qualidade agrupa vários atributos específicos a serem implementados no componente de software.

Os fatores de qualidade do software podem ser divididos em dois grupos:

- Fatores que podem ser medidos diretamente. Exemplo: defeitos por ponto de função.
- Fatores que podem ser medidos indiretamente. Exemplo: usabilidade ou manutenibilidade.

1.3.2 Atributo da qualidade

O atributo da qualidade representa uma característica particular do produto, passível de ser medida e avaliada. Um ou vários atributos fornecem valores que **permitem avaliar um determinado fator de qualidade.**

– Sistema de software de gestão empresarial (ERP) um dos principais atributos da qualidade é a funcionalidade.

Algumas das seguintes métricas são consideradas no sistema de software de gestão empresarial:

- Usabilidade (RNF 1 – USAB): medição do índice de satisfação do usuário, de qualquer perfil, nas operações com o sistema de software; nível de implementação de requisitos que atendem às necessidades do usuário.
- Implementação (RNF 2 – IMPL): acompanhamento do cronograma, observando as metas referentes a prazos e custos.
- Integração com outras tecnologias (RNF 3 – INTI): medições de índice de integração da nova tecnologia com a já existente na empresa; grau de portabilidade para outros sistemas de software.
- Nível de atualizações (RNF 4 – NVAT): um baixo índice de atualizações indica robustez do projeto de software

1.3.3 Métricas da qualidade

Métricas são padrões de medidas dimensionadas para quantificar e qualificar o resultado de um determinado processo, produto ou serviço. A medida atende a um determinado padrão que expressa o resultado quantitativo de um atributo específico, seu formato e tipo de dados.

As dimensões das medidas estão relacionadas aos resultados das medições. Por exemplo: comprimento (metro – m); período (segundos – s); massa (grama – g); dígito (bit – b) e diversas outras. Elas podem ser combinadas e associadas para formar um resultado específico. Por exemplo: velocidade em m/s; armazenamento em bytes (1 byte = 8 bits) e taxa de transferência em bps (ou bit/s). Os dados coletados são analisados por engenheiros de software, comparados com médias anteriores e avaliados para verificar se ocorreu algum desvio no processo de software.

Quadro 2 – Principais métricas de qualidade de software

Métrica	Definição
Acurácia	Capacidade do produto de software de prover, com o grau de precisão necessário, resultados ou efeitos corretos ou conforme acordados
Adaptabilidade	Capacidade do produto de software de ser adaptado para diferentes ambientes especificados, sem necessidade de aplicação de outras ações ou meios além daqueles fornecidos para essa finalidade pelo software considerado
Adequação	Capacidade do produto de software de prover um conjunto apropriado de funções para tarefas e objetivos do usuário especificados
Analisabilidade	Capacidade do produto de software de permitir o diagnóstico de deficiências ou causas de falhas e a identificação de partes a serem modificadas
Apreensibilidade	Capacidade do produto de software de possibilitar ao usuário o aprendizado de sua aplicação
Atratividade	Capacidade do produto de software de ser atraente ao usuário
Auditabilidade	Facilidade no atendimento às normas que pode ser verificado
Autodocumentação	Nível em que o código-fonte fornece documentação significativa
Capacidade	Capacidade do produto de software de ser instalado em ambiente especificado e atender aos requisitos de funcionamento
Coexistência	Capacidade do produto de software de coexistir com outros produtos de software independentes, em um ambiente comum, compartilhando recursos comuns
Confiabilidade	A efetiva realização das funções de um programa em um nível de desempenho e precisão especificadas
Conformidade	Capacidade do produto de software de estar de acordo com normas, convenções ou regulamentações previstas em leis e prescrições similares relacionadas ao atributo do software
Eficácia	Capacidade do produto de software de permitir que usuários atinjam metas especificadas com acurácia e completitude, em um contexto de uso especificado
Eficiência	Quantidade de recursos e códigos de computação para realização da função. Quanto menor a quantidade, maior será o desempenho do software
Estabilidade	Capacidade do produto de software de evitar efeitos inesperados decorrentes de modificações no software
Expansibilidade	Nível em que o projeto arquitetural de dados ou procedimental pode ser estendido
Flexibilidade	Esforço necessário para modificar um programa operacional
Funcionalidade	Capacidade do produto de software de prover funções que atendam às necessidades explícitas e implícitas, quando o software estiver sendo utilizado sob condições especificadas
Generalidade	Âmbito da aplicação potencial dos componentes do programa
Integridade	Diz respeito ao acesso do software ou de dados por pessoas não autorizadas que pode ser controlado
Interoperabilidade	Capacidade do produto de software de interagir com um ou mais sistemas especificados

Métrica	Definição
Mantenabilidade	Esforço necessário para localizar e corrigir erros num programa
Manutenabilidade	Capacidade do produto de software de ser modificado. As modificações podem incluir correções, melhorias ou adaptações do software devido a mudanças no ambiente e nos seus requisitos ou especificações funcionais
Maturidade	Capacidade do produto de software de evitar falhas decorrentes de defeitos no software
Modificabilidade	Capacidade do produto de software de permitir que uma modificação especificada seja implementada
Modularidade	Diz respeito à independência funcional dos componentes do programa
Precisão	Nível de precisão dos cálculos e do controle que pode ser verificado
Operacionalidade	Capacidade do produto de software de possibilitar ao usuário operá-lo e controlá-lo
Produtividade	Capacidade do produto de software de permitir que seus usuários empreguem quantidade apropriada de recursos em relação à eficácia obtida, em um contexto de uso especificado
Portabilidade	Capacidade do produto de software de ser transferido de um ambiente operacional para outro
Rastreabilidade	Diz respeito à habilidade de rastrear uma representação de projeto ou componente real do programa
Recuperabilidade	Capacidade do produto de software de restabelecer seu nível de desempenho especificado e recuperar os dados diretamente afetados no caso de uma falha
Reutilização	Quanto de um programa ou parte dele pode ser reutilizado em outras aplicações
Satisfação	Capacidade do produto de software de satisfazer usuários, em um contexto de uso especificado Nota: satisfação é a resposta do usuário à interação com o produto e inclui atitudes relacionadas ao uso do produto
Segurança	Capacidade do produto de software de apresentar níveis aceitáveis de riscos de danos às pessoas, aos negócios, da propriedade, do ambiente, de proteger informações e dados, de modo que pessoas ou sistemas não autorizados não possam lê-los nem modificá-los e que não seja negado o acesso às pessoas ou sistemas autorizados
Testabilidade	É necessário testar um programa para sua validação, quando criado ou modificado, a fim de garantir que realize a função esperada
Usabilidade	Capacidade do produto de software de ser compreendido, aprendido, operado e atrativo ao usuário, quando usado sob condições especificadas
Utilização	Esforço necessário para aprender, operar, preparar entradas e interpretar saídas de um programa

Exemplo de aplicação

Para **determinar as métricas da qualidade** são considerados os atributos da **qualidade do requisito funcional** gerenciamento do BD: **implementação (IMPL)**, **integração com outras tecnologias (INTI)** e **nível de atualizações (NVAT)**.

- Implementação (IMPL): medida direta de produção, em períodos de dias (d.), comparada com as metas definidas no cronograma.
- Integração com outras tecnologias (INTI): podem ser efetuadas medidas indiretas, tais como medidas de portabilidade do BD, que definem na prática os sistemas operacionais e outros gerenciadores de BDs em que o software irá operar.
- Nível de atualizações (NVAT): são medidas diretas de versionamento do software que indicam o índice de mudanças no código-fonte. São definidas as versões que acompanham o desenvolvimento do software e as versões liberadas ao cliente. O número de mudanças em um determinado período pode indicar instabilidade do software.

2 ENGENHARIA DE SEGURANÇA DE SOFTWARE

A engenharia de segurança de software planeja, desenvolve e mantém sistemas seguros e confiáveis, protegendo dados e funcionalidades contra falhas, vulnerabilidades e ataques. Tem como objetivo garantir a integridade, confidencialidade, autenticidade e disponibilidade das informações ao longo de todo o ciclo de vida do software.

As técnicas aplicadas fornecem práticas como análise de ameaças, modelagem de riscos, testes de penetração e criptografia

2.1 Gerenciamento e segurança: análise do risco, plano de contingência e seguro

Gerenciamento de risco (GR), o plano de contingência (PC) e a seguridade (seguro) aplicados ao software envolvem práticas sistemáticas que visam identificar, avaliar e mitigar riscos que possam comprometer a qualidade, a continuidade e a confiabilidade dos sistemas computacionais, para a sustentabilidade operacional e a proteção dos ativos de informação.

O risco é conceituado como “um evento ou condição incerta que, se ocorrer, tem um efeito positivo ou negativo sobre ao menos um dos objetivos a ser cumprido”, ou seja, a probabilidade de perigo, uma ameaça física para o homem ou para o meio em que se encontra, ou a probabilidade de insucesso em função de um acontecimento ou incidente que acarrete indenização.

A análise do risco consiste na identificação, categorização e avaliação das ameaças e vulnerabilidades que podem afetar o software, considerando tanto aspectos técnicos (falhas de código, ataques cibernéticos, indisponibilidade de servidores) quanto organizacionais (erros humanos, falhas de comunicação, requisitos mal definidos).

Elementos	Definições
Ameaça	Qualquer evento, pessoa ou condição que possa causar um resultado indesejado ou comprometer um artefato ou o processo de software. Exemplo: atraso no fornecimento de uma API de fornecedor terceirizado
Probabilidade	É a chance de uma ameaça se concretizar durante o desenvolvimento de uma funcionalidade, sprint ou o comprometimento com o ciclo de entregas. Exemplo: há grande probabilidade de que a implementação de um novo código contenha regressões, se testes unitários não forem concluídos
Impacto	É a medida do dano ou custo (em tempo, dinheiro, reputação ou funcionalidade) que a ocorrência do risco pode acarretar ao produto de software, ao desempenho da equipe e/ou ao compromisso de entregas. Exemplo: impacto da mudança na ocorrência de uma falha de software que impede a implantação do software
Incerteza	O grau de incerteza é medido de acordo com a falta de informações claras sobre um determinado requisito, uma dependência técnica ou a viabilidade de uma solução. Nas metodologias ágeis, isso é frequente e conhecido como spike (em inglês: atividade de pesquisa ou experimentação técnica), com objetivo de converter a incerteza em informação
Ação alternativa	É a estratégia de mitigação ou plano de contingência (rollback, que significa ação corretiva) definido para neutralizar ou reduzir o risco. Exemplo: implementar um recurso de fallback (recuo) para serviços de terceiros, ou refatorar o código para reduzir o débito técnico

Plano de contingência, um documento estratégico que define ações preventivas e corretivas para garantir a continuidade do serviço e a recuperação dos sistemas após incidentes. Inclui rotinas de backup e recuperação de dados, estratégias de redundância de servidores, planos de resposta a incidentes de segurança e mecanismos de failover (processo automático de transferência de operações e dados entre servidores), para minimizar o tempo de inatividade e garantir a continuidade dos serviços. Trata-se de práticas de DevOps e de integração contínua/entrega contínua, que facilitam a rápida correção de falhas, com objetivo de reduzir a exposição a riscos.

O **seguro** aplicado a software (ou seguro cibernético) surge como uma ferramenta financeira de mitigação de risco, destinada a cobrir prejuízos decorrentes de incidentes de segurança, como vazamento de dados, interrupção de serviços, ataques de ransomware e falhas de infraestrutura.

2.2 Gerenciamento de riscos do projeto

O risco pode ser conceituado como a combinação da probabilidade da ocorrência de um evento e de suas consequências sobre os objetivos do projeto. Em termos quantitativos, o risco pode ser representado pela relação:

Risco = Probabilidade (P) × Impacto (I)

- Probabilidade (P): o quanto é provável que o risco ocorra (por exemplo: baixa, média ou alta).
- Impacto (I): quão severas serão as consequências, caso o risco ocorra (por exemplo: pequeno, moderado ou crítico).

Nos modelos prescritivos (cascata, espiral e outros), o gerenciamento do risco é aplicado em fases específicas do ciclo de desenvolvimento, enquanto nas metodologias ágeis (Scrum, XP, Kanban, FDD e outros), o monitoramento e a mitigação de riscos ocorrem de forma iterativa, ou seja, promovem ajustes rápidos ao longo do ciclo de desenvolvimento da funcionalidade ou da sprint.

Principais processos de gerenciamento dos riscos do projeto:

- 1)Planejar o gerenciamento do risco: definir como as atividades de risco serão conduzidas no projeto, considerando também a dinâmica das metodologias ágeis.
- 2)Identificar os riscos: levantar riscos técnicos, de negócio e de processo por meio de reuniões de planejamento, histórias de usuário (user story) e retrospectivas.
- 3)Realizar análise qualitativa e quantitativa dos riscos: priorizar riscos (baixo, médio e alto), conforme seu impacto (pequeno, moderado ou crítico) nas mudanças e/ou nas entregas de curto prazo.
- 4)Planejar as respostas aos riscos: definir estratégias como spikes técnicos, automação de testes e pair programming para mitigar incertezas.
- 5)Implementar respostas aos riscos: aplicar as ações planejadas de mitigação e contingência durante as iterações. Em um ambiente ágil, isso ocorre durante as sprints, com a equipe ajustando atividades, prioridades e recursos.

2.3 Governança em TI

A governança em tecnologia da informação (TI) se baseia em um conjunto de estratégias, políticas para uso da tecnologia, estruturas e processos decisórios com o intuito de apoiar e expandir os objetivos estratégicos e operacionais da

organização. Frameworks como COBIT 2019 e ITIL v4 desempenham papéis fundamentais, fornecendo diretrizes para a governança e a gestão de serviços de TI. COBIT 2019 possui fatores de design que permitem a personalização do sistema de governança de TI. O Control Objectives for Information and Related Technologies (COBIT) é um framework para gestão e controle de ambientes de TI empresariais, alinhando as estratégias de negócios com as estratégias de TI. ITIL v4, por sua vez, enfatiza a gestão de serviços de TI com foco na entrega de valor e na melhoria contínua. O Information Technology Infrastructure Library (ITIL) integra formas de trabalho como lean, metodologias ágeis e DevOps.

2.4 Estratégia para suporte de software

Ela deve englobar a gestão proativa de riscos, a otimização da experiência do usuário (UX) e a manutenção da satisfação do cliente a longo prazo.

Quadro 4 – Princípios de integração da engenharia de segurança e metodologias ágeis

Princípio	Objetivo	Prática
Segurança tratada como RNF	A segurança é tratada como parte da qualidade do software	Definir histórias de usuário com critérios de segurança (security stories)
Resiliência e continuidade	O sistema deve suportar falhas (bugs) e ataques sem perda crítica	Implementar mecanismos de rollback (reverter), redundância de código e dados e logs de auditoria
Iteratividade e resposta rápida	Entregas curtas favorecem correções e reforços de segurança contínuos	Aplicar DevSecOps e pipelines de CI/CD com testes de segurança automatizados
Cultura de responsabilidade compartilhada	A segurança é de toda a equipe de desenvolvimento, incluindo cliente e usuários, não apenas especialista	Pair programming e revisões de código com foco em vulnerabilidades

2.5 Retorno de investimento (ROI) no ciclo de entrega ágil

O retorno de investimento (ROI) é observado diretamente na eficácia da aplicação e na sua capacidade de suportar os objetivos estratégicos da organização.

ROI é maximizado pela adoção de práticas ágeis que garantem que o capital investido se traduza rapidamente em resultados mensuráveis como:

- Minimizar custo de refatoração e feedbacks rápidos: consiste na entrega de sprints em ciclos curtos, permitindo que stakeholders validem as funcionalidades de forma rápida, reduzindo, assim, o risco de desenvolver um recurso errado, economizando tempo e recursos.

- Aumento da produtividade e eficiência operacional: um software que atende com precisão e clareza aos requisitos de negócio se traduz diretamente em maior velocidade de adoção, redução de erros operacionais

2.6 Métodos, ferramentas e técnicas (MF&T): melhoria da qualidade de software

Plano de garantia da qualidade (GQS), contemplando: • padrões de codificação baseados em Cleanroom; • revisões de código (peer review) obrigatórias via GitHub; • checklist em conformidade com ISO/IEC 25010 (funcionalidade, confiabilidade, desempenho e segurança); • auditorias internas de qualidade a cada três sprints.

Métricas de qualidade (ferramentas aplicáveis)

No quadro a seguir são apresentadas algumas das ferramentas aplicáveis para mensurar a evolução da qualidade:

Quadro 5 – Métricas da qualidade e ferramentas aplicáveis

Métrica	Ferramenta/desenvolvedor	Características
Eficiência	Jira/Atlassian	Mede o tempo entre a criação de uma tarefa e sua conclusão Eficiência na remoção de defeitos antes da entrega; permite o acompanhamento de bugs, tarefas e qualidade do processo
Eficiência	GitHub – modo Kanban/ GitHub, Inc. (Microsoft)	Mede o throughput (taxa de conclusão) Tempo médio para corrigir falhas Acompanhamento visual do fluxo de trabalho
Desempenho	GanttProject/ Bartłomiej Nawrocki	Geração de cronogramas, distribuição de responsabilidades e tarefas (ou sprints) Checklist; medição de velocidade da sprint (story points entregues)

Quadro 6 – Metas a serem atingidas

Categoria	Meta da qualidade	Métrica/medida	Tipo de custo
Eficiência do processo	Reduzir em 20% o tempo médio de resolução de tarefas em três sprints	Lead time	Prevenção e avaliação
Remoção de defeitos antes da entrega	Aumentar em 30% a taxa de correção de defeitos detectados internamente, antes da entrega ao cliente	Taxa de defeitos removidos antes da entrega	Falhas internas
Produtividade da equipe	Elevar a velocidade média da sprint em 10%, mantendo a qualidade estável	Story points entregues/sprint	Avaliação
Confiabilidade pós-entrega	Diminuir em 25% as falhas externas reportadas nos dois primeiros meses após a entrega	Quantidade de bugs reportados pós-entrega	Falhas externas